

## Problem Set 4

---

Due Date ..... October 01, 2024  
Name ..... Hyma Jujjuru  
Student ID ..... 110743269  
Collaborators ..... List Your Collaborators Here

### Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Instructions</b>                                      | <b>1</b>  |
| <b>Honor Code (Make Sure to Virtually Sign) [10 pts]</b> | <b>2</b>  |
| <b>1 Graph traversals</b>                                | <b>3</b>  |
| 1.1 Problem 1 [15 pts]                                   | 3         |
| 1.2 Problem 2 [15 pts]                                   | 5         |
| <b>2 Shortest Path Algorithms</b>                        | <b>8</b>  |
| 2.1 Problem 3 [15 pts]                                   | 8         |
| 2.2 Problem 4 [30 pts]                                   | 11        |
| 2.2.a Problem 2(a)                                       | 11        |
| 2.2.b Problem 2(b)                                       | 12        |
| 2.2.c Problem 2(c)                                       | 13        |
| <b>3 Minimum spanning tree</b>                           | <b>16</b> |
| 3.1 Problem 5 [15 pts]                                   | 16        |

### Instructions

- The solutions **should be typed**, using proper mathematical notation. We cannot accept hand-written solutions. Here's a short intro to  $\text{\LaTeX}$ .
- You should submit your work through the **class Gradescope page** only (linked from Canvas). Please submit one PDF file, compiled using this  $\text{\LaTeX}$  template.
- You may not need a full page for your solutions; pagebreaks are there to help Gradescope automatically find where each problem is. Even if you do not attempt every problem, please submit this document with no fewer pages than the blank template (or Gradescope has issues with it).

- You are welcome and encouraged to collaborate with your classmates, as well as consult outside resources. You must **cite your sources in this document**. **Copying from any source is an Honor Code violation. Furthermore, all submissions must be in your own words and reflect your understanding of the material.** If there is any confusion about this policy, it is your responsibility to clarify before the due date.
- Posting to **any** service including, but not limited to Chegg, Reddit, StackExchange, etc., for help on an assignment is a violation of the Honor Code.
- You **must** virtually sign the Honor Code (see Section ). Failure to do so will result in your assignment not being graded.

## Honor Code (Make Sure to Virtually Sign) [10 pts]

**Problem HC.**     • My submission is in my own words and reflects my understanding of the material.

- Any collaborations and external sources have been clearly cited in this document.
- I have not posted to external services including, but not limited to Chegg, Reddit, StackExchange, etc.
- I have neither copied nor provided others solutions they can copy.

*Agreed (signature here).* I agree to the above, Hyma Jujjuru.

□

# 1 Graph traversals

## 1.1 Problem 1 [15 pts]

**Problem 1.** In the epic world of comics, There are two teams: Heroes and Villains. Each character may have rivalries with others, leading to intense conflicts. Given a list of characters from both universes and pairs that indicate rivalries, your task is to create an algorithm that determines if it's possible to assign each character to either the Heroes or the Villains such that:

- Every rivalry exists between a Hero and a Villain.
- No two characters from the same group have a rivalry.

### Requirements:

- Design an efficient algorithm that decides whether such an assignment is possible.
- If it is possible, your algorithm should produce the assignment of characters to their respective groups.
- Analyze the time complexity of your solution.

Provide a brief explanation of your algorithm's logic. Discuss any potential edge cases and how your algorithm addresses them.

| Rubric                                                           | Points           |
|------------------------------------------------------------------|------------------|
| Correct logic and algorithm with pseudocode                      | 6 Points         |
| Time complexity analysis                                         | 3 Points         |
| Correctness explanation                                          | 3 Points         |
| Handling edge cases (e.g., empty graph, disconnected components) | 3 Point          |
| <b>Total</b>                                                     | <b>15 Points</b> |

*Proof.* This problem can be viewed as a graph problem where the nodes are the characters and the edges are the rivalries between Heroes and Villains. Also, the characters can be assigned to either the Heroes or the villains such that a Hero and a Villain share an edge and no two characters (no two Heroes or no two Villains) within a group share an edge. This is behavior that represents a bipartite graph.

### Algorithm:

To make sure all the characters can be assigned to either the Heroes or the Villains, we need to check whether the graph is bipartite. To do so, we need to check that the graph does not have an odd cycle.

Details of function:

- The function takes in a graph of vertices and edges (or characters and rivalries respectively in this case) and a source vertex as parameters.
- The color array holds values where 0 represents red for Hero nodes, 1 represents blue for Villain nodes, and -1 represents that no color was assigned to that node.
- Returns false if an odd cycle exists, so the requested assignment is impossible.
- Returns false if the q is empty because no nodes are pushed into the queue when the graph is empty.
- Returns false if there is an unvisited node.
- Returns true if an odd cycle does not exist, so the requested assignment is possible and the nodes are colored with their respective group's color.

```

Assign(G(V,E), s):
    initialize an integer colorArr[V] to -1
    initialize visited[V] to False
    initialize queue q
    q.push(s) //q will be empty if the graph is empty
    if(q is empty) return false
    while(q is not empty){
        curr = q.pop()
        for(x : adj[curr]){ //check neighbors
            if(colorArr[x] == -1){
                x = 1 - colorArr[curr] // if curr is 0, then x is 1.
                q.push(x)
            }
            elseif(colorArr[curr] == colorArr[x]){
                return false
            }
        }
    }
    if(visited[V] is not all true) return false

    return true

```

### Correctness Explanation:

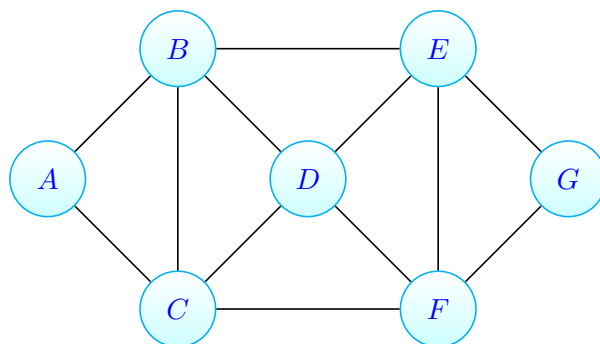
The algorithm determines whether the requested assignment is possible by coloring the nodes to divide them into their respective group.

- **Loop Invariant:** If the requested assignment is possible, the order in which the nodes are colored is always  $colorArr[i] \neq colorArr[j]$  such that  $j$  is a neighbor of  $i$ .
- **Initiation:** At the first iteration, there is only one element in the queue: the starting vertex,  $A$ . So the invariant holds initially because there are no vertices/neighbors in the queue to compare the color of  $A$ .
- **Maintenance:** At the start of some iteration, assume the vertices in the queue are  $v_1, v_2, \dots, v_k$ . When  $v_1$  is dequeued from the queue, its unvisited neighbors  $n_1, n_2, \dots, n_i$  where  $i = 0, \dots, k$  are enqueued to the queue. All of the neighbors are colored in the opposite color of  $v_1$  because  $x = 1 - colorArr[curr]$  which means if  $v_1$  is color 0(*red*) then its neighbors will be  $1 - 0 = 1$ (*blue*) or if  $v_1$  is color 1(*blue*) then its neighbors will be  $1 - 1 = 0$ (*red*). If the color of  $v_1$  and any of its neighbors have the same color, the requested assignment is not possible and the algorithm returns true.
- **Termination:** The requested assignment is only possible when the graph is bipartite where no odd cycle exists. In other words, if a vertex and its neighbor(s) are colored the same color, the requested assignment is not possible. Else, the requested assignment is possible and all vertices are colored such that vertices connected by an edge/rivalry have opposite colors(0 vs 1 or red vs blue).

□

## 1.2 Problem 2 [15 pts]

**Problem 2.** Consider the unweighted graph  $G(V, E)$ . List the order that the nodes are considered by BFS and DFS when starting from node  $A$ . **Use the ordering that nodes are added to the queue (stack) alphabetically.**



| Rubric                                                  | Points           |
|---------------------------------------------------------|------------------|
| Correct BFS node order from node $A$                    | 3 Points         |
| Correct DFS node order from node $A$                    | 5 Points         |
| Clear explanation of BFS (queue, alphabetical ordering) | 4 Points         |
| Clear explanation of DFS (stack, alphabetical ordering) | 3 Points         |
| <b>Total</b>                                            | <b>15 Points</b> |

*Answer.* Given the graph above,

- **BFS: A Breadth First Search** traversal gives the shortest path of an unweighted graph and uses a queue.

1. Initialize the queue  $Q$  and `visited[x]` array such that all nodes  $x$  are set to false/unvisited.

$$Q = []$$

|         |   |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|---|
| Node    | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| Visited | F | F | F | F | F | F | F |

2. Enqueue the starting node, node  $A$  in this case, and mark it as visited (True).

$$Q = [A]$$

|         |     |   |   |   |   |   |   |
|---------|-----|---|---|---|---|---|---|
| Node    | $A$ | 1 | 2 | 3 | 4 | 5 | 6 |
| Visited | T   | F | F | F | F | F | F |

3. Dequeue  $A$  and enqueue its unvisited neighbors  $B$  and  $C$  marking them as visited.

$$\text{After popping } A: Q = []$$

$$\text{After enqueue unvisited neighbors: } Q[B, C]$$

4. Mark each node as visited when enqueued.

|                |     |     |     |   |   |   |   |
|----------------|-----|-----|-----|---|---|---|---|
| <b>Node</b>    | $A$ | $B$ | $C$ | 3 | 4 | 5 | 6 |
| <b>Visited</b> | T   | T   | T   | F | F | F | F |

5. Dequeue  $B$  and enqueue  $B$ 's unvisited neighbors( $D, E$ ).

$$\text{After popping } B: Q = [C]$$

$$\text{After enqueue unvisited neighbors: } Q[C, D, E]$$

6. Mark each node as visited when enqueued.

|                |     |     |     |     |     |   |   |
|----------------|-----|-----|-----|-----|-----|---|---|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | 5 | 6 |
| <b>Visited</b> | T   | T   | T   | T   | T   | F | F |

7. Dequeue  $C$  and enqueue  $C$ 's unvisited neighbors( $F$ ).

After popping  $C$  :  $Q = [D, E]$

After enqueue unvisited neighbors:  $Q[D, E, F]$

8. Mark each node as visited when enqueued.

|                |     |     |     |     |     |     |   |
|----------------|-----|-----|-----|-----|-----|-----|---|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ | 6 |
| <b>Visited</b> | T   | T   | T   | T   | T   | T   | F |

9. Dequeue  $D$  and enqueue  $D$ 's unvisited neighbors( $none$ ).

After popping  $D$  :  $Q = [E, F]$

10. There are no unvisited neighbors of  $D$ , so none are marked as visited.

|                |     |     |     |     |     |     |   |
|----------------|-----|-----|-----|-----|-----|-----|---|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ | 6 |
| <b>Visited</b> | T   | T   | T   | T   | T   | T   | F |

11. Dequeue  $E$  and enqueue  $E$ 's unvisited neighbors( $G$ ).

After popping  $E$  :  $Q = [F]$

After enqueue unvisited neighbors:  $Q[F, G]$

12. Mark each node as visited when enqueued.

|                |     |     |     |     |     |     |     |
|----------------|-----|-----|-----|-----|-----|-----|-----|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ | $G$ |
| <b>Visited</b> | T   | T   | T   | T   | T   | T   | T   |

13. Dequeue  $F$  and enqueue  $F$ 's unvisited neighbors( $none$ ).

After popping  $F$  :  $Q = [G]$

14. There are no unvisited neighbors of  $F$ , so none are marked as visited.

|                |     |     |     |     |     |     |     |
|----------------|-----|-----|-----|-----|-----|-----|-----|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ | $G$ |
| <b>Visited</b> | T   | T   | T   | T   | T   | T   | T   |

15. Dequeue  $G$  and enqueue  $G$ 's unvisited neighbors( $none$ ).

After popping  $G$  :  $Q = []$

16. There are no unvisited neighbors of  $G$ , so none are marked as visited.

|                |     |     |     |     |     |     |     |
|----------------|-----|-----|-----|-----|-----|-----|-----|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ | $G$ |
| <b>Visited</b> | T   | T   | T   | T   | T   | T   | T   |

**BFS Complete:** All nodes are visited and the queue's memory is cleared.

- **DFS: A Depth First Search** traverses to the farthest node and uses a stack data structure.

1. Initialize the stack  $S$  and **visited**[ $x$ ] array such that all nodes  $x$  are set to false/unvisited.

$S = []$

|         |   |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|---|
| Node    | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| Visited | F | F | F | F | F | F | F |

2. Push the starting node, node  $A$  in this case, to the top of the stack and mark it as visited (True).

$S = [A]$

|         |     |   |   |   |   |   |   |
|---------|-----|---|---|---|---|---|---|
| Node    | $A$ | 1 | 2 | 3 | 4 | 5 | 6 |
| Visited | T   | F | F | F | F | F | F |

3. Check for unvisited neighbors. Push one out of the unvisited neighbors  $B, C$  to the stack. We push  $B$  because we push alphabetically.

Push  $B$ :  $S = [A, B]$

4. Mark  $B$  as visited.

|                |     |     |   |   |   |   |   |
|----------------|-----|-----|---|---|---|---|---|
| <b>Node</b>    | $A$ | $B$ | 2 | 3 | 4 | 5 | 6 |
| <b>Visited</b> | T   | T   | F | F | F | F | F |

5. We push one unvisited neighbor of  $B$  choosing alphabetically. SO we push  $C$ .

Push  $C$ :  $S = [A, B, C]$

6. Mark  $C$  as visited.

|                |     |     |     |   |   |   |   |
|----------------|-----|-----|-----|---|---|---|---|
| <b>Node</b>    | $A$ | $B$ | $C$ | 3 | 4 | 5 | 6 |
| <b>Visited</b> | T   | T   | T   | F | F | F | F |

7. We push one unvisited neighbor of  $C$  alphabetically. So we choose  $D$ .

Push  $D$ :  $S = [A, B, C, D]$

8. Mark  $D$  as visited.

|                |     |     |     |     |   |   |   |
|----------------|-----|-----|-----|-----|---|---|---|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | 4 | 5 | 6 |
| <b>Visited</b> | T   | T   | T   | T   | F | F | F |

9. We push one unvisited neighbor of  $D$  alphabetically. So we choose  $E$ .

Push  $E$ :  $S = [A, B, C, D, E]$

10. Mark  $E$  as visited.

|                |     |     |     |     |     |   |   |
|----------------|-----|-----|-----|-----|-----|---|---|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | 5 | 6 |
| <b>Visited</b> | T   | T   | T   | T   | T   | F | F |

11. We push one unvisited neighbor of  $E$  alphabetically. So we choose  $F$ .

Push  $F$ :  $S = [A, B, C, D, E, F]$

12. Mark  $F$  as visited.

|                |     |     |     |     |     |     |   |
|----------------|-----|-----|-----|-----|-----|-----|---|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ | 6 |
| <b>Visited</b> | T   | T   | T   | T   | T   | T   | F |

13. We push one unvisited neighbor of  $G$  alphabetically. So we choose  $G$ .

Push  $G$ :  $S = [A, B, C, D, E, F, G]$

14. Mark  $G$  as visited.

|                |     |     |     |     |     |     |     |
|----------------|-----|-----|-----|-----|-----|-----|-----|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ | $G$ |
| <b>Visited</b> | T   | T   | T   | T   | T   | T   | T   |

15. There are no unvisited neighbors of  $G$ , so none are marked as visited.

|                |     |     |     |     |     |     |     |
|----------------|-----|-----|-----|-----|-----|-----|-----|
| <b>Node</b>    | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ | $G$ |
| <b>Visited</b> | T   | T   | T   | T   | T   | T   | T   |

**DFS Complete:** All nodes are visited and the stack's memory is cleared.

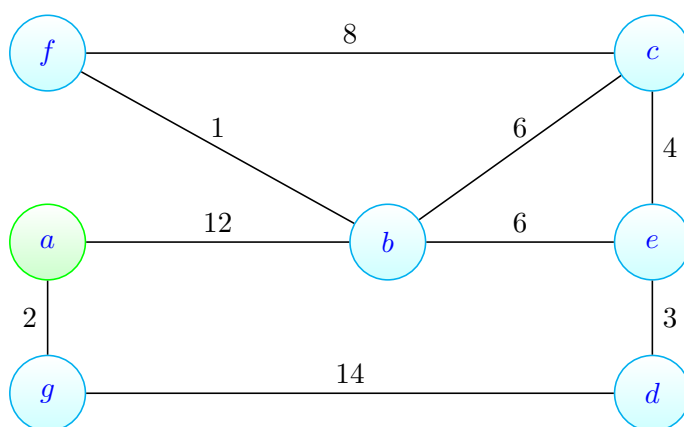
□

## 2 Shortest Path Algorithms

### 2.1 Problem 3 [15 pts]

**Problem 3.** Consider the undirected weighted graph  $G(V, E, w)$  pictured below. Work through Dijkstra's algorithm on the following graph, using the **source vertex**  $a$ . **Note:** In order to get full credits consider the following:

- Clearly include the contents of the priority queue, the distance from  $a$  and the parent of each vertex at each iteration.
- If you use a table to store the distances, clearly label the keys according to the vertex names rather than numeric indices (i.e., `dist['B']` is more descriptive than `dist[1]`).
- You do **not** need to draw the graph at each iteration, though you are welcome to do so. [This may be helpful scratch work, which you do not need to include.]
- Finally represent the shortest path graph.



| Rubric                                                                     | Points           |
|----------------------------------------------------------------------------|------------------|
| Accurate execution and iteration of Dijkstra's algorithm                   | 5 Points         |
| Proper updating of distances, parents, and management of priority queue    | 4 Points         |
| Clear presentation of state (distances, parents, queue) for each iteration | 2 Point          |
| Correct labeling of vertices (letters instead of indices)                  | 2 Point          |
| Final correct shortest path graph representation                           | 2 Points         |
| <b>Total</b>                                                               | <b>15 Points</b> |

*Answer.* Dijkstra's algorithm uses a minimum priority queue to find the shortest path from the source vertex,  $s = a$ , to the destination vertex.

1. Initialize  $Q = []$
2. Enqueue the source vertex,  $(a, 0)$ , mark as visited, and update table.

$$Q = [(a, 0)]$$

| Node          | $a$ | $b$      | $c$      | $d$      | $e$      | $f$      | $g$      |
|---------------|-----|----------|----------|----------|----------|----------|----------|
| Dist from $a$ | 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| Parent        | —   | —        | —        | —        | —        | —        | —        |
| Visited       | T   | F        | F        | F        | F        | F        | F        |



3. Dequeue  $(a, 0)$  and enqueue  $a$ 's unvisited neighbors,  $(g, 2)$ ,  $(b, 12)$ , mark as visited, and update table.

After dequeue:  $Q = []$   
 After enqueue unvisited neighbors:  $Q = [(g, 2), (b, 12)]$

| Node          | $a$ | $b$ | $c$      | $d$      | $e$      | $f$      | $g$ |
|---------------|-----|-----|----------|----------|----------|----------|-----|
| Dist from $a$ | 0   | 12  | $\infty$ | $\infty$ | $\infty$ | $\infty$ | 2   |
| Parent        | —   | $a$ | —        | —        | —        | —        | $a$ |
| Visited       | T   | T   | F        | F        | F        | F        | T   |

4. Dequeue  $(g, 2)$  and enqueue  $g$ 's unvisited neighbors,  $(d, 14)$ , mark as visited, and update table.

After dequeue:  $Q = [(b, 12)]$   
 After enqueue unvisited neighbors:  $Q = [(b, 12), (d, 14)]$   
 $dist[a \rightarrow d] = dist[a \rightarrow g] + dist[g \rightarrow d] = 2 + 14 = 16$

| Node          | $a$ | $b$ | $c$      | $d$ | $e$      | $f$      | $g$ |
|---------------|-----|-----|----------|-----|----------|----------|-----|
| Dist from $a$ | 0   | 12  | $\infty$ | 16  | $\infty$ | $\infty$ | 2   |
| Parent        | —   | $a$ | —        | $g$ | —        | —        | $a$ |
| Visited       | T   | T   | F        | T   | F        | F        | T   |

5. Dequeue  $(b, 12)$  and enqueue  $b$ 's unvisited neighbors,  $(f, 1)$ ,  $(c, 6)$ ,  $(e, 6)$ , mark as visited, and update table.

After dequeue:  $Q = [(d, 14)]$   
 After enqueue unvisited neighbors:  $Q = [(d, 14), (f, 1), (c, 6), (e, 6)]$   
 $dist[a \rightarrow f] = dist[a \rightarrow b] + dist[b \rightarrow f] = 12 + 1 = 13$   
 $dist[a \rightarrow c] = dist[a \rightarrow b] + dist[b \rightarrow c] = 12 + 6 = 18$   
 $dist[a \rightarrow e] = dist[a \rightarrow b] + dist[b \rightarrow e] = 12 + 6 = 18$

| Node          | $a$ | $b$ | $c$ | $d$ | $e$ | $f$ | $g$ |
|---------------|-----|-----|-----|-----|-----|-----|-----|
| Dist from $a$ | 0   | 12  | 18  | 16  | 18  | 13  | 2   |
| Parent        | —   | $a$ | $b$ | $g$ | $b$ | $b$ | $a$ |
| Visited       | T   | T   | T   | T   | T   | T   | T   |

6. Dequeue  $(d, 14)$  and enqueue  $d$ 's unvisited neighbors, *none*, mark as visited, and no updates made to the table.

After dequeue:  $Q = [(f, 1), (c, 6), (e, 6)]$

7. Dequeue  $(f, 1)$  and enqueue  $f$ 's unvisited neighbors, *none*, mark as visited, and no updates made to the table.

After dequeue:  $Q = [(c, 6), (e, 6)]$

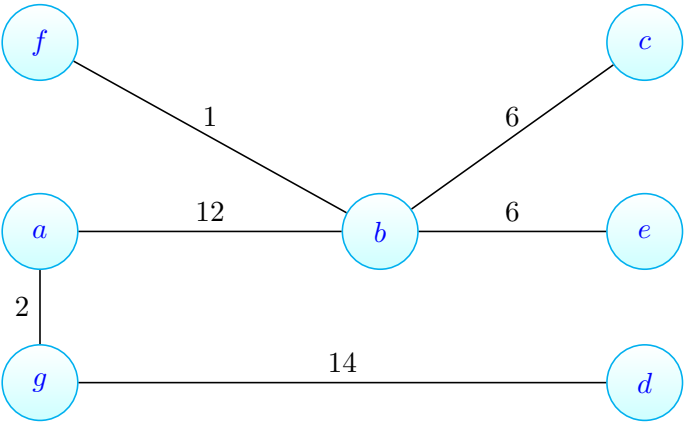
8. Dequeue  $(c, 6)$  and enqueue  $c$ 's unvisited neighbors, *none*, mark as visited, and no updates made to the table.

After dequeue:  $Q = [(e, 6)]$

9. Dequeue  $(e, 6)$  and enqueue  $e$ 's unvisited neighbors, *none*, mark as visited, and no updates made to the table.

After dequeue:  $Q = []$

10. Shortest path graph:



□

## 2.2 Problem 4 [30 pts]

**Problem 4.** You have three batteries, with capacities of 40, 25, and 16 Ah (Amp-hours), respectively. The 25 and 16-Ah batteries are fully charged (containing 25 Ah and 16 Ah, respectively), while the 40-Ah battery is empty, with 0 Ah. You have a battery transfer device which has a “source” battery position and a “target” battery position. When you place two batteries in the device, it instantaneously transfers as many Ah from the source battery to the target battery as possible. Thus, this device stops the transfer either when the source battery has no Ah remaining or when the destination battery is fully charged (whichever comes first).

But battery transfers aren’t free! The battery device is also hooked up to your phone by bluetooth, and automatically charges you a number of dollars equal to however many Ah it just transferred.

The goal in this problem is to determine whether there exists a sequence of transfers that leaves exactly 10 Ah either in the 25-Ah battery or the 16-Ah battery, and if so, how little money you can spend to get this result.

| Rubric                                                                        | Points           |
|-------------------------------------------------------------------------------|------------------|
| <b>Part (a) - Graph Modeling</b> (5 points)                                   |                  |
| Correct definition of vertices and edges                                      | 4 Points         |
| Clear statement of the graph problem                                          | 1 Point          |
| <b>Part (b) - Algorithm Description</b> (5 points)                            |                  |
| Appropriate algorithm choice and modifications                                | 4 Points         |
| Correct specification of initial parameters                                   | 1 Point          |
| <b>Part (c) - Algorithm Application</b> (5 points)                            |                  |
| Correct application, graph states, sequence of vertices, and cost calculation | 5 Points         |
| <b>Total</b>                                                                  | <b>15 Points</b> |

Do the following.

### 2.2.a Problem 2(a)

- (a) Rephrase this as a graph problem. Give a precise definition of how to model this problem as a graph, and state the specific question about this graph that must be answered. [**Note:** While you are welcome to draw the graph, it is enough to provide 1-2 sentences clearly describing what the vertices are and when two vertices are adjacent. If the graph is weighted, clearly specify what the edge weights are.]

*Answer.* The graph represents all the possible combinations of charges the three batteries can hold and the transfers that can occur.

The states of the three batteries represent the vertices of the graph.

The weight of the edges depends on the sequence of transfers. The weight represents the amount of Ah being transferred from each battery which is equivalent to the cost of the transfer.

For instance, the 40Ah battery is empty and the 25Ah and 16Ah batteries are full. So, the edge between 40Ah and 25AH would be 25 and cost \$25 and the edge between 40Ah and 16Ah would be 16 and \$16.

**Question:** What sequence of transfers leaves exactly 10Ah in either the 25Ah battery or the 16 Ah battery while minimizing the cost? □

### 2.2.b Problem 2(b)

- (b) Clearly describe an algorithm to solve this problem. If you use an algorithm covered in class, it is enough to state that. If you modify an algorithm from class, clearly outline any modifications. Make sure to explicitly specify any parameters that need to be passed to the initial function call. You need not write the algorithm.

*Answer.* The algorithm used is similar to Dijkstra's.

Considerations:

- The battery capacity is the maximum that can be stored in each vertex (i.e. can only have 40Ah max in 40Ah battery)
- During each transfer, the total current Ah in the source battery is completely transferred into the target battery until the source battery is empty or the target battery is full.
- the initial function call should take in a graph  $G(V,E)$  where  $V$  is the list of the states of the batteries and  $E$  is the list of edges between the vertices, a source vertex which is the initial state of batteries (0 Ah, 25 AH, 16Ah), weights of the graph (amount transferred between batteries), and the capacities of the batteries.

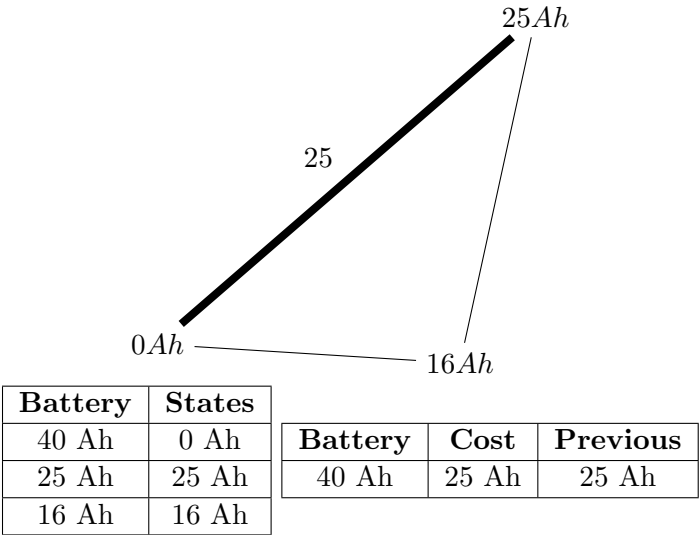
So, while checking if the shortest/cheapest path is already found, we also check if the node is at capacity. If the node is not at capacity and is cheaper than the path that was already found then we can add that edge to the path. We also need to check if the 25Ah battery or the 16Ah battery has reached 10Ah. If either battery reaches 10Ah, then we have reached our goal and are completed with our transfers.  $\square$

2.2.c Problem 2(c)

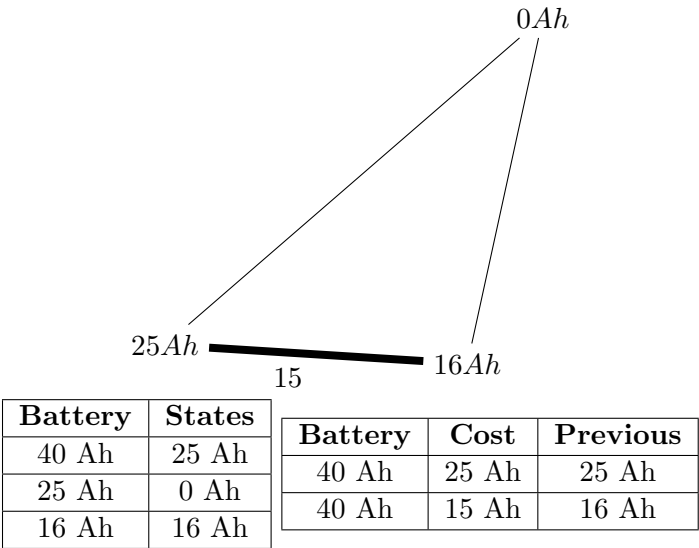
- (c) Apply that algorithm to the question. Report and justify your answer. Here, justification includes the sequences of vertices visited and the total cost.
- (**Note:** For full credits make sure to draw the graph with all the battery states until the final states. **No need to draw the entire graph.** Then apply the algorithm that you have described until the desired final states are reached.)

*Answer.* Applying the algorithm to the question:  
Initial state of batteries: (0Ah, 25Ah, 16Ah) for the 40Ah, 25Ah, 16Ah batteries respectively.

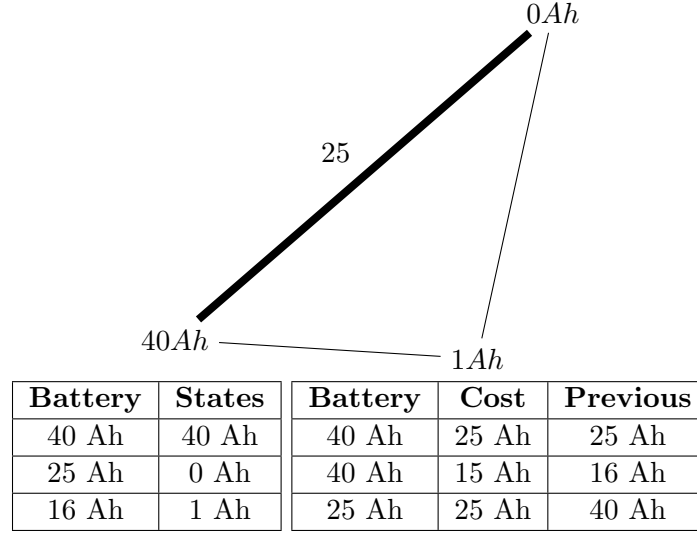
Transfer: 25 Ah  $\longrightarrow$  40Ah 25 Ah is transferred from the 25 Ah battery to the 40 Ah battery. the 25 Ah battery is empty.



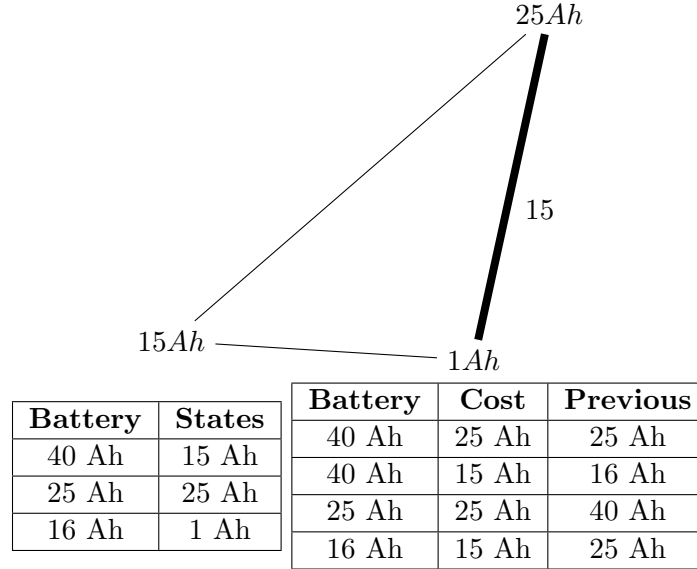
Transfer: 16 Ah  $\longrightarrow$  40Ah. 15 Ah is transferred and the 40 Ah battery is full. There is 1 Ah left in the 16 Ah battery.



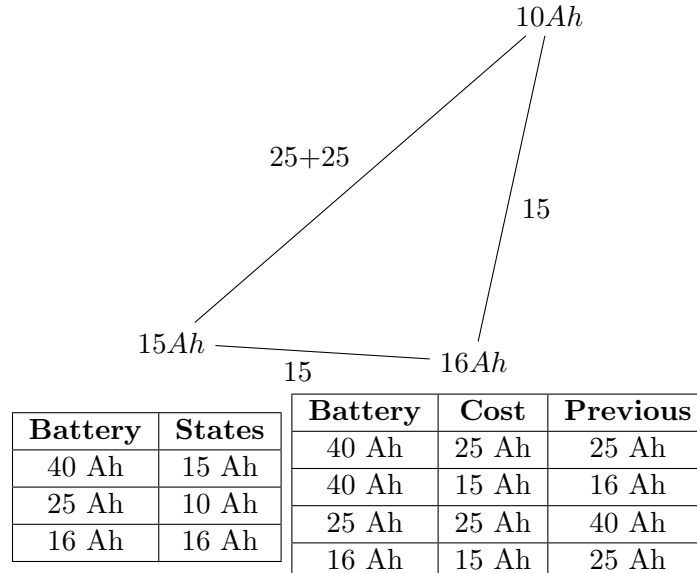
Transfer: 40 Ah  $\rightarrow$  25Ah. 25 Ah is transferred from the 40 Ah battery to the 25 Ah battery and the 40 Ah battery is full. There is 15 Ah left in the 40 Ah battery.



Transfer: 25 Ah  $\rightarrow$  16Ah. 15 Ah transferred from the 25 Ah battery to the 16 Ah battery.



Final State: The 25 Ah battery has reached 10 Ah so we can stop transferring.



**Total Cost:**  $25 + 25 + 15 + 15 = \$80$

**Sequence of Transfers:** 25 Ah, 40 Ah, 16 Ah, 40 Ah, 25 Ah, 16 Ah

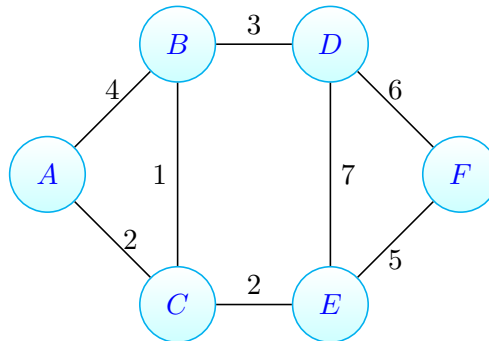
While all the possible combinations and outcomes are not presented, Dijkstra's algorithm can be used to compare paths to find the shortest path for the problem even with restrictions placed on the graph such as the considerations above.  $\square$

### 3 Minimum spanning tree

#### 3.1 Problem 5 [15 pts]

##### Problem 5.

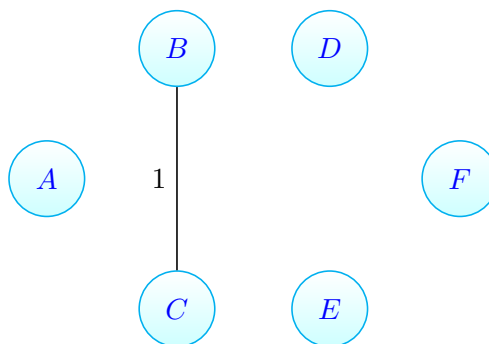
**Problem 6.** Consider the following graph  $G(V, E, w)$ . Show the steps that Kruskal's algorithm takes in finding a min spanning tree. For each step, specify: what edge is being considered, and why the algorithm adds or doesn't add it to the MST. (When an edge would create a cycle, specify the edges in the cycle.)



| Rubric                                                                        | Points           |
|-------------------------------------------------------------------------------|------------------|
| Correct ordering of edges and step-by-step application of Kruskal's algorithm | 6 Points         |
| Clear explanation for each edge addition, rejection, and cycle detection      | 6 Points         |
| Correct final Minimum Spanning Tree                                           | 3 Point          |
| <b>Total</b>                                                                  | <b>15 Points</b> |

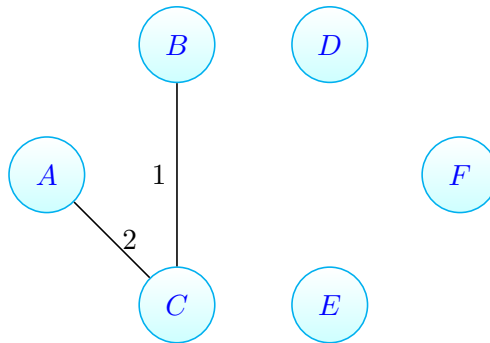
*Answer.* **Kruskal's Algorithm:**

- Sort the edges from minimum to maximum based on weights  
Sorted edges:  $(B, C) : 1, (A, C) : 2, (C, E) : 2, (B, D) : 3, (A, B) : 4, (E, F) : 5, (D, F) : 6, (D, E) : 7$
- Pick the edge and add it to the tree such that it does not create a cycle
  - Edge:  $(B, C) : 1$   
Conflict: None  $\rightarrow$  Add to MST

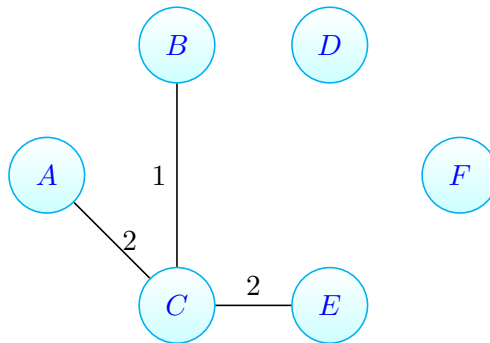




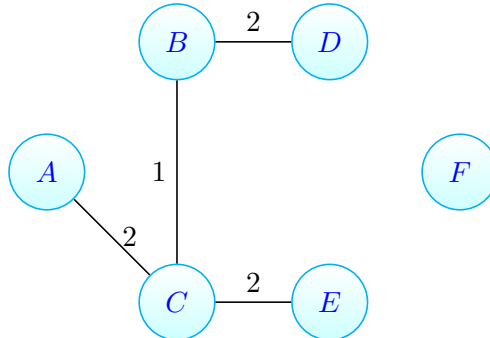
- Edge:  $(A, C) : 2$   
Conflict: None  $\longrightarrow$  Add to MST



- Edge:  $(C, E) : 2$   
Conflict: None  $\longrightarrow$  Add to MST

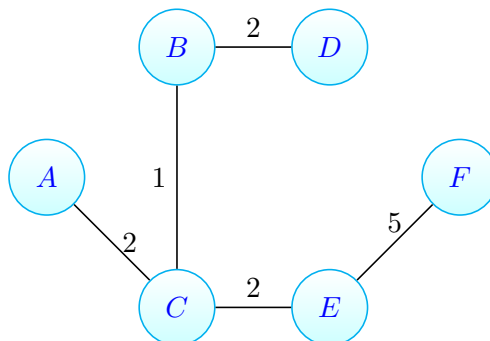


- Edge:  $(B, D) : 3$   
Conflict: None  $\longrightarrow$  Add to MST



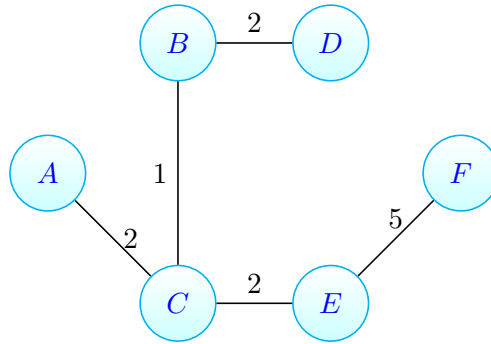
- Edge:  $(A, B) : 4$   
Conflict: Cycle  $\longrightarrow$  Do not add to MST  
Creates a cycle between nodes  $A, B, C$   
Edges in the cycle:  $(B, C) : 1, (A, C) : 2, (A, B) : 4$

- Edge:  $(E, F) \Rightarrow 5$   
Conflict: None  $\longrightarrow$  Add to MST



- Edge:  $(D, F) : 6$   
 Conflict: Cycle  $\rightarrow$  Do not add to MST  
 Creates a cycle between nodes  $B, D, F, E, C$   
 Edges in the cycle:  $(B, C) : 1, (C, E) : 2, (B, D) : 3, (E, F) : 5, (D, F) : 6$
- Edge:  $(D, E) : 7$   
 Conflict: Cycle  $\rightarrow$  Do not add to MST  
 Creates a cycle between nodes  $B, D, E, C$   
 Edges in the cycle:  $(B, C) : 1, (C, E) : 2, (B, D) : 3, (D, E) : 7$

### 3. Final Minimum Spanning Tree:



□